

Python Minecraft Server Manager Blueprint

Overview: This blueprint outlines a cross-platform Python application for configuring and running Minecraft servers. It features a step-by-step GUI (e.g. built with PySide6/PyQt6) for selecting the server loader and version, Java settings, and optional Docker deployment. Key functions include an embedded console (for real-time logs and commands via the RCON protocol ¹ ²), automated first-run setup (generating `eula.txt`, `server.properties`, and ops files), and mod/plugin management through the CurseForge and Modrinth APIs ³ ⁴. The design is modular, with separate components for the UI, server control, data storage, and external APIs, ensuring extensibility (e.g. adding new mod loaders).

Core Architecture: The application follows a Model-View-Controller (MVC) style. The **UI Layer** (view) is built with Qt (using PySide6/PyQt6 for native look-and-feel across Windows, macOS, and Linux ⁵). A **Control Layer** handles server startup/stop via `subprocess` or Qt's `QProcess`, and RCON commands using libraries like `mcipc` or `mcrcon` ¹ ⁶. The **Data Layer** manages user settings and profiles (stored in JSON/YAML) and persistent server configurations. An **API Layer** interfaces with external services: the PaperMC Downloads API for Paper/Velocity servers ⁷, Mojang's version manifest for Vanilla servers ⁸, and Minecraft mod/plugin services (CurseForge and Modrinth) via their Python SDKs ³ ⁴. A **Platform Abstraction** module detects the OS/architecture (`sys.platform`, `platform.machine()`) and adjusts paths and packaging.

- **Dependencies:** PySide6/PyQt6 (or PySide2/PyQt5) for GUI; `requests` for HTTP API calls; `mcipc` / `mcrcon` for RCON; `docker` (docker-py) for Docker support ⁹; `curseforge` and `modrinth` Python libraries for mod queries ³ ⁴; `json` / `yaml` for configs; `platform` and `sys` for OS detection; and tools like PyInstaller for bundling.

- **Packaging:** Use PyInstaller (or py2app) to build executables. PyInstaller must be run on each target OS ("you must install PyInstaller on each platform and bundle your app separately on each" ¹⁰) to produce native packages for Windows (.exe/.msi), macOS (.app), and Linux.

GUI Setup Wizard

1. **Startup Wizard:** The app launches a multi-page Qt Wizard guiding the user through server setup. Pages include:
2. **Loader Selection:** Choose among Vanilla, Paper, Spigot/Bukkit, Forge, Fabric, Quilt, Mohist, etc. The app queries the chosen loader's version API to populate available Minecraft versions/builds ⁷ ⁸.
3. **Version Picker:** After selecting loader, fetch the list of supported game versions. For example, for PaperMC use `https://api.papermc.io/v2/projects/paper` to get recent versions ⁷; for Vanilla use Mojang's version manifest (at `https://piston-meta.mojang.com/v1/versions/java/` ⁸). For Spigot/Bukkit, the tool invokes BuildTools to compile a jar (SpigotMC distributes only source to comply with Mojang's license ¹¹).
4. **Java Check:** Detect installed Java (`subprocess.check_output(['java', '-version'], stderr=subprocess.STDOUT)` ¹²). If insufficient, prompt to install a proper JDK (e.g. adoptOpenJDK or Azul).

5. **Docker Option:** Offer running the server in Docker. If chosen, use the Docker SDK (`docker` Python package) to pull or build a Minecraft server image ⁹ . Otherwise, plan for a native `java -jar server.jar` run.
6. **EULA & Basics:** Show the EULA and require user consent. Collect key `server.properties` settings (port, gamemode, difficulty, etc.) and initial ops list.
7. **UI Details:** The GUI uses Qt Widgets (or QtQuick) for a polished interface ⁵ . Controls include dropdowns for versions, checkboxes for options (online-mode, whitelist, etc.), and file dialogs for custom mod selection. Use `QWizard` or stacked `QWidget` s for the multi-step flow. At the end, a summary page shows the chosen configuration.
8. **Embedded Console:** The main window includes a console panel (e.g. a `QPlainTextEdit`) to show server stdout/stderr in real-time. Instead of emulating a system terminal, the app connects to the running server via RCON to issue commands and stream logs ¹ ² . The console input box (a `QLineEdit`) sends commands through an RCON client (like `mcipc.rcon` or `mcrcon` ⁶). This avoids OS-specific terminal embedding issues – RCON is “the best way” to run commands on a Minecraft server ¹ . (In the background, the server process has RCON enabled with a randomly generated password.)

Server Loaders Support

- **Vanilla:** Retrieve versions from Mojang’s official manifest (piston-meta) ⁸ . Download the `server.jar` for the selected version and place it in the server folder.
- **PaperMC/Velocity:** Use the PaperMC Downloads API. For example, GET `https://api.papermc.io/v2/projects/paper/versions/{mc_version}/builds` to find the latest stable build, then download via their API ⁷ .
- **Spigot/Bukkit:** Invoke Spigot’s BuildTools (a bundled Maven build) to generate the CraftBukkit/Spigot jar. As noted, SpigotMC “downloads/compiles the jars” via BuildTools ¹¹ due to licensing. The GUI can run BuildTools in the background or use a precompiled version if allowed.
- **Forge:** Query the Forge Maven or CurseForge for installer downloads. (One approach: use the CurseForge API to find the “forge” project and retrieve the server installer jar ³ .) Alternatively, present known Forge versions and download from `files.minecraftforge.net` .
- **Fabric:** Use the Fabric installer JAR (offered on Fabric’s site) to generate a Fabric server. The app can download the Fabric-installer jar and run it headlessly with chosen MC and loader version.
- **Quilt:** Similar to Fabric, use Quilt’s installer or repository. Provide quilt loader versions from their metadata.
- **Mohist (Hybrid Forge+Bukkit):** Download from MohistMC’s releases (no public API). The app can scrape `https://mohistmc.com/` or GitHub to list versions.
- **Others:** The tool should be extensible to new loaders; each loader has a handler module (e.g. `PaperLoader` , `SpigotLoader` , etc.) that knows how to fetch versions and download the server jar.

All loader version fetching is done dynamically via HTTP. For example, PaperMC’s v2 API “facilitates automated downloads access” ⁷ , and Mojang’s manifest is updated with every release ⁸ . Python’s `requests` library is used to query these endpoints and parse JSON.

Mod/Plugin Integration

- **APIs:** Use the official APIs via Python wrappers. The [CurseForge Python library](#) is a “no-compromise CurseForge API wrapper” ³. The [Modrinth.py](#) library provides access to Modrinth’s API ⁴. These let users search for mods/plugins by keyword or category, filter by game version and loader, and fetch download URLs.
- **Functionality:** The GUI includes a mod browser pane. Under the covers, a `ModManager` module uses the CurseForge and Modrinth clients. For example, to find mods compatible with Fabric 1.20, call the Modrinth API endpoint for loader=“fabric” and version=“1.20”. The APIs return lists of projects and file downloads which the app can display (with name, description, author, icon).
- **Installation:** When the user selects mods/plugins, the app downloads the chosen JAR files and places them into the server’s `mods/` or `plugins/` folder. It also tracks installed versions to allow easy updates (e.g. compare local file hashes against latest available).
- **Citing:** Both libraries handle API communication. For instance, `curseforge` lets you “list all the games” and then fetch a mod’s files ³; `modrinth` can “get project versions” and search for projects by queries ⁴. Using these avoids manual REST calls and JSON parsing.

Java Environment Management

- **Detection:** On startup, the app runs `java -version` via `subprocess.check_output` (capturing stderr) to detect any installed Java JRE/JDK ¹². It parses the output (e.g. regex `"(\d+\.\d+)"`) to get the major version. If no Java is found, or the version is too low (e.g. Minecraft 1.20+ requires Java 17+), the GUI prompts the user.
- **Installation/Download:** The app can guide the user to install Java manually, or (if permissions allow) download an appropriate JRE. For example, it might fetch an OpenJDK archive from an adoptopenjdk API. After installation, it updates its Java executable path in settings.
- **Path Handling:** On different platforms, Java executables differ (`java.exe` on Windows, `java` on Unix). The code uses `shutil.which("java")` and `platform.system()` to find the executable. For embedded JREs (e.g. for portability), the app could bundle a Java runtime, but licensing must be respected.

Docker Option

- **Docker SDK:** If the user opts for a Docker container, the app uses the official Docker Engine SDK for Python (the `docker` PyPI package) ⁹. The SDK “lets you do anything the `docker` command does” from Python – running containers, managing images, etc. It can pull a community Minecraft server image (like `itzg/minecraft-server`), or build a Dockerfile on the fly with the chosen server jar.
- **Workflow:** The GUI asks if Docker should be used. If yes, ensure Docker is installed/running (check `docker info`). Then use the Docker SDK: e.g. `docker.client.images.pull("itzg/minecraft-server:latest")` and `client.containers.run(...)` with the selected configuration. Logs from the container are streamed back into the embedded console.
- **Fallback:** If Docker is not available or the user prefers native, the app simply starts `java -jar` as usual.

First-Run Configuration

When a new server is set up and started for the first time:

- **Generate Files:** Running the server jar once will auto-generate the basic files (`server.properties`, `eula.txt`, `whitelist.json`, `ops.json`, etc.) in the server directory. The app then stops the server process.
- **Accept EULA:** The blueprint automatically writes `eula.txt` with `eula=true` if the user agreed in the GUI. (Minecraft requires this text change to run, as a legal acknowledgment.)
- **server.properties:** The app updates key properties (port, gameMode, difficulty, max-players, etc.) based on the user's setup inputs. For example, it might set `enable-rcon=true` and write a generated `rcon.password` ¹³. (This allows the GUI to use RCON immediately.) The properties file is standard `key=value` text.
- **Add Ops:** If the user provided player names or UUIDs for operators, the app appends them to `ops.json` (in JSON format) or runs the equivalent RCON command (`/op username`).
- **Notes:** All these file writes happen before the server is started again. The app can open the `server.properties` file with a minimal UI editor for additional tweaks.

Server Lifecycle & Persistence

- **Profiles:** Each server configuration (loader, version, mods list, `server.properties` settings) is saved as a profile (e.g. in `~/minecraft_server_manager/servers/`). Users can create multiple profiles (e.g. "Survival World", "Modpack Test") and switch between them. Profiles store metadata (paths, settings, mod lists).
- **Commands:** From the main UI or CLI, the user can start, stop, or restart the server. Starting executes `java -Xmx... -jar server.jar nogui` via `QProcess` or `subprocess.Popen` (managed asynchronously). Stop sends `/stop` via RCON or terminates the process.
- **Logs:** Standard output from the server process is captured and shown live in the GUI console. Errors and important messages are also saved to a log file per server instance (for later review).
- **Updates:** The app can check for updates to the server loader itself (e.g. a new Paper build). It compares current version to API listings and offers one-click upgrade. Similarly, it checks installed mods/plugins against CurseForge/Modrinth for updates.
- **Persistence:** All settings (e.g. last window size, recent servers) are stored using a lightweight database or config files. For cross-session persistence, SQLite or JSON are sufficient. This lets the user close the app and later resume managing any server.

Third-Party Components & APIs

- **PySide6/PyQt6:** Provides the cross-platform GUI toolkit ⁵. The app uses Qt Designer for layout and PySide signals/slots for events.
- **requests:** For HTTP calls to version/loader APIs.
- **mcipc/mcrcon:** For RCON communication ¹ ², allowing real-time command execution and log retrieval without OS-dependent terminal hacks.
- **docker (docker-py):** To manage Docker containers if the Docker option is enabled ⁹.
- **curseforge & modrinth:** Python libraries to query and download mods/plugins ³ ⁴. They simplify integrating the CurseForge API (which requires an API key) and Modrinth's public API.
- **PyInstaller or similar:** For bundling into executables. (Remember to build on each OS ¹⁰.)
- **Other:** Standard libraries (`os`, `sys`, `subprocess`, `json`, `platform`, `threading` / `asyncio`). Optionally, `PyYAML` if YAML config is preferred.

Modules and Code Structure

A suggested modular layout (Python package):

- `ui/` – Qt Designer `.ui` files and generated Python UI classes. Contains `main_window.py`, `wizard_pages.py`, etc.
- `controllers/` – Application logic classes, e.g. `ServerController` (start/stop server), `ModManager` (handle mod/plugin installation), `JavaManager` (detect/install Java), `DockerManager`.
- `models/` – Data models for server profiles and settings (could be simple classes or dataclasses).
- `loaders/` – Classes for each server loader (e.g. `PaperLoader`, `SpigotLoader`, `ForgeLoader`) with methods `fetch_versions()`, `download_server_jar()`.
- `apis/` – Wrappers for external APIs if needed (e.g. a small wrapper around PaperMC API, Mojang version manifest, or use direct `requests` calls here).
- `utils/` – Utility functions (e.g. `run_process`, `parse_version`, `detect_os`, `write_config`).
- `assets/` – Any static resources (icons, default config templates).
- `main.py` – The entry point that initializes the application and shows the GUI.

This separation keeps the codebase organized: the UI layer imports controller classes, which use loader classes and APIs as needed. For example, when the user clicks “Download Paper 1.20 Server”, the UI calls `PaperLoader.download(version)` which handles HTTP requests as documented ⁷.

Platform Compatibility

- **Windows:** Executables built with PyInstaller (onefile or onefolder). Use native path separators and registry check for Java (or bundled JRE). If `os.startfile` is needed for opening folders. Installer (MSI/EXE) can be created.
- **macOS:** Bundle as `.app` using PyInstaller or py2app. Code sign or notarize if distributed widely. Use `.dmg` installer. Be mindful of case-sensitivity and Unix permissions.
- **Linux:** Provide AppImage or packaged binary (e.g. deb/rpm via fpm). Ensure `chmod +x` for jar files. The path `/` vs `~`.
- **General:** Detect architecture (x86_64 vs arm64) via `platform.machine()` and handle Java vendor differences (for example, arm builds require a compatible JDK).
- **Packaging:** Each platform build installs required dependencies (Qt libs on target). Recommend shipping Qt `.dll/.so/.dylib` with the app bundle so the user need not install Qt separately.

Conclusion

This blueprint combines established tools and APIs to create a full-featured Minecraft server manager in Python. It leverages Qt for a rich GUI (cross-platform by design ⁵), RCON libraries for live server control ¹ ², and online services (PaperMC, CurseForge, Modrinth) to automate tedious setup steps. By modularizing loader support and using Python’s standard libraries (with clear division between UI, control, and data layers), the app remains maintainable and extensible. Users get a one-stop utility to configure, launch, and manage servers and mods without manually editing files or using the command line.

Sources: Official APIs and libraries underpin each component. For example, PaperMC’s downloads API ⁷ and Mojang’s version manifest ⁸ ensure accurate version lists; RCON libraries like *mcrcon* ¹ /

mcipc ² enable robust command execution; and the CurseForge/Modrinth Python wrappers ³ ⁴ simplify mod discovery. Packaging guidance is from PyInstaller's documentation ¹⁰ . These sources inform the recommended architecture and tool choices.

- ¹ ¹³ **Controlling a minecraft server with python - Stack Overflow**
<https://stackoverflow.com/questions/40750727/controlling-a-minecraft-server-with-python>
- ² **mcipc · PyPI**
<https://pypi.org/project/mcipc/>
- ³ **curseforge · PyPI**
<https://pypi.org/project/curseforge/>
- ⁴ **modrinth · PyPI**
<https://pypi.org/project/modrinth/>
- ⁵ **Which GUI Framework is the best for Python coders?**
<https://www.activestate.com/blog/top-10-python-gui-frameworks-compared/>
- ⁶ **mcrcon · PyPI**
<https://pypi.org/project/mcrcon/>
- ⁷ **Downloads API | PaperMC Docs**
<https://docs.papermc.io/misc/downloads-api/>
- ⁸ **version_manifest.json – Minecraft Wiki**
https://minecraft.fandom.com/wiki/Version_manifest.json
- ⁹ **Docker SDK for Python - Read the Docs**
<https://docker-py.readthedocs.io/>
- ¹⁰ **Using PyInstaller — PyInstaller 6.13.0 documentation**
<https://pyinstaller.org/en/stable/usage.html>
- ¹¹ **linux - Downloading/Buidling minecraft server jars using BuildTools in Bash/Shell script - Stack Overflow**
<https://stackoverflow.com/questions/43911743/downloading-buidling-minecraft-server-jars-using-buildtools-in-bash-shell-script>
- ¹² **bash - Get java version number from python - Stack Overflow**
<https://stackoverflow.com/questions/31807882/get-java-version-number-from-python>